

MALVIYA NATIONAL INSTITUTE OF
TECHNOLOGY

OPERATING SYSTEMS PROJECT

(22CST912)

BENZENE: A Practical Root Cause Analysis System with an Under-Constrained State Mutation

Authors of paper:

Younggi Park

Hwiwon Lee

Jinho Jung

Hyungjoon Koo

Huy Kang Kim

Report submitted by:

Yash Kumar Jain

2023UCP1571

Abstract

In this document, we study the research paper “BENZENE: A Practical Root Cause Analysis System with an Under-Constrained State Mutation,” published at the IEEE Symposium on Security and Privacy (S&P) 2024. The paper introduces BENZENE, an automated root cause analysis (RCA) system designed to localize the root causes of software crashes. BENZENE employs a novel technique called *under-constrained state mutation*, which bypasses complex input-processing branches by directly mutating the program state at runtime. The system was evaluated on 60 real-world bugs across diverse applications and outperformed state-of-the-art tools both in accuracy (93.3%) and speed (approximately $8\times$ faster than ARCUS). This report summarizes the paper’s methodology, describes implementation challenges encountered while exploring the tool, and reflects on its broader significance.

Contents

1	Introduction	3
2	Motivation	3
3	Methodology	4
3.1	Crash Analysis and Target Function Extraction	4
3.2	Under-Constrained State Mutation	4
3.3	Behavior Collection and Predicate Extraction	4
3.4	Root Cause Ranking	5
3.5	Evaluation Setup	5
4	Challenges during Exploration	5
5	Results and Discussion	6
6	Conclusion	6

1 Introduction

Software crashes are a pervasive problem in systems security. When a crash occurs—whether discovered through fuzzing or in a deployed application—developers need to understand *why* it happened before they can fix it. This process, known as Root Cause Analysis (RCA), is time-consuming and requires deep expertise.

Existing approaches to RCA typically rely on predicate-based fault localization: they collect program behaviors from both crashing and non-crashing executions, extract statistical predicates describing when crashes occur, and rank those predicates by suspiciousness to identify the root cause. However, generating the non-crashing behaviors required by these approaches is extremely difficult. Finding inputs that are semantically similar to the crashing input but do not trigger the crash often requires manually crafting inputs or solving complex path constraints—neither of which scales well in practice.

This paper introduces BENZENE, a system published at IEEE S&P 2024 that automates RCA via an *under-constrained state mutation* strategy. Instead of searching for crash-similar non-crashing inputs in the input space, BENZENE directly mutates the program’s in-memory state at the entry of a crash-related function, bypassing complex input handlers entirely. The mutated executions then serve as non-crashing behavior samples for predicate extraction and root cause ranking.

BENZENE is evaluated on 60 real-world bugs from applications such as PHP, Python, SQLite, and PDF readers, correctly localizing the root cause in 56 out of 60 cases. The tool is open-source and available at <https://github.com/zer0fall/BENZENE>.

2 Motivation

The primary motivation behind BENZENE is the fundamental bottleneck in existing RCA systems: the difficulty of obtaining crash-similar yet non-crashing program behaviors. To understand the crashing condition of a bug, an RCA system must observe how the program behaves when it *does not* crash but follows a path very close to the crashing one. Existing tools either rely on fuzzing—which may fail to reach the buggy code due to complex input-parsing branches—or on constraint solvers, which are computationally expensive and brittle.

Consider CVE-2019-6977, a heap buffer overflow in PHP’s `gdImageColorMatch` function. The function allocates a buffer of size `0x28 * colorsTotal`, but when `colorsTotal` is very small (e.g., `0x1`) and the pixel color value is large (e.g., `0x80`), the buffer pointer overflows into unallocated memory, causing a crash. To localize this, an RCA system needs a non-crashing execution where `colorsTotal` is sufficiently large. However, generating such an input requires setting up a PHP script that allocates 255 different palette colors—a non-trivial crafting effort that cannot easily be automated through fuzzing alone.

BENZENE’s insight is that this non-crashing behavior can be obtained by simply mutating the register holding `colorsTotal` at the entry of `gdImageColorMatch` to a sufficiently large value, without changing the input at all. This approach is both practical and fast, which motivated the development of under-constrained state mutation as the core technique.

My motivation for studying this paper was to understand how dynamic binary analysis and in-memory fuzzing concepts could be combined to solve a real security engineering problem, and to ex-

plore whether the open-source implementation matched the claimed results.

3 Methodology

BENZENE implements a four-stage pipeline to perform automated root cause analysis. The pipeline is built on top of Intel Pin, a dynamic binary instrumentation framework, and uses `fork()`-based execution to efficiently generate behavior samples.

3.1 Crash Analysis and Target Function Extraction

When given a crashing input, BENZENE first performs a dynamic analysis of the crashing execution. It records the crash site and traces back the data-flow and dereference chains to identify *target functions*—those functions whose local states are most likely related to the crash. This is done by following data-flow edges backward from the crashing instruction and identifying functions that supply values to the crashing computation. Only these crash-related functions are selected for state mutation, significantly narrowing the search space.

3.2 Under-Constrained State Mutation

For each selected target function, BENZENE suspends execution at the function’s entry point and applies random mutations to the program state. The mutated state includes:

- **Register values** – modifying general-purpose registers that carry function parameters or externally derived values.

- **Memory values** – mutating memory locations pointed to by registers, specifically those holding data from outside the function (i.e., global variables or heap-allocated buffers filled by callers).
- **Focused mutation scope** – only *entry node* values are mutated, since a function’s behavior is entirely determined by the state at its entry. Internal computations are left to execute naturally.

The mutation is “under-constrained” in the sense that it does not enforce all the invariants that a normally-constructed program state would satisfy. This is intentional: for the purposes of extracting a crashing condition, behavioral reachability does not need to be guaranteed in the same way as for bug discovery. What matters is that the extracted predicates correctly describe the conditions under which the crash occurs.

A `fork()`-based design is used to avoid restarting the entire program for each mutation trial. The process is forked at the function entry, the child applies the mutation and resumes execution, and the parent waits for the result. This design is analogous to in-memory fuzzing techniques used in tools like LibFuzzer and AFL’s persistent mode.

3.3 Behavior Collection and Predicate Extraction

Each mutated execution either crashes or exits cleanly. BENZENE records the values of key program variables (parameters, global variables, memory contents) at the target function entry for both outcomes. These constitute the *crashing behavior dataset*: a set of labeled samples annotated with whether the execution crashed.

From this dataset, BENZENE extracts statistical predicates by comparing crash-

ing and non-crashing sample distributions. For a scalar variable v , it identifies boundary values—the maximum value at which the program still crashes and the minimum value at which it exits cleanly—and formulates a predicate such as $v < \theta$ for some threshold θ . This predicate-based approach is rooted in the Cooperative Bug Isolation (CBI) methodology.

3.4 Root Cause Ranking

Once predicates are extracted for all observed variables across all target functions, BENZENE ranks them using a similarity-based scoring algorithm. Predicates that are highly correlated with the crashing outcome and appear across many samples are ranked higher. The top-ranked predicate is presented as the most likely root cause of the crash, along with its associated variable, location in the binary, and the corresponding source-level context when debug symbols are available.

3.5 Evaluation Setup

The system was evaluated on a dataset of 60 bugs manually collected from real-world programs including PHP, Python, SQLite, MuPDF, and others. The bugs span 11 classes including heap overflow, integer overflow, use-after-free (UAF), null pointer dereference, and stack overflow. BENZENE was compared against two state-of-the-art RCA systems, AURORA and ARCUS, on overlapping subsets of the dataset. BENZENE achieved 93.3% accuracy (56/60 bugs correctly localized) compared to 63% for ARCUS (12/34) and 32% for AURORA (29/46), while running approximately 8× faster than ARCUS with a mean elapsed time of 1,567 seconds versus 12,762 seconds.

4 Challenges during Exploration

- **Build environment sensitivity.** BENZENE relies on Intel Pin and specific versions of system libraries. Setting up the correct environment required careful version pinning of Pin, GCC, and target application builds. Any mismatch resulted in silent instrumentation failures rather than clear error messages.
- **Understanding the mutation harness.** The repository contains multiple modules covering crash analysis, target function extraction, state mutation, and predicate ranking. Understanding the interaction between these components—particularly how the fork-based child processes communicate crash/non-crash outcomes back to the parent—required careful reading of both the paper and the source code together.
- **Reproducing the crashing inputs.** The paper uses 60 real-world CVEs. Obtaining the exact crashing proof-of-concept (PoC) scripts for each and building the vulnerable version of the target application (e.g., a specific PHP 7.x release from 2019) was tedious, as many build scripts are no longer directly compatible with modern Linux distributions.
- **Interpreting predicate output.** The tool outputs ranked predicates in a binary-level representation (register names, addresses). Mapping these back to source-level variable names required cross-referencing with debug symbols and, in some cases, manually inspecting the disassembly. A source-level pretty-printer would significantly improve usability.

5 Results and Discussion

The key result of the paper is that BENZENE successfully localizes the root cause in 56 out of 60 real-world bugs (93.3% accuracy), significantly outperforming both AURORA (32%) and ARCUS (63%). Beyond accuracy, BENZENE’s median analysis time of approximately 26 minutes per bug is practical for real-world use, whereas ARCUS requires over 3.5 hours on average.

The four cases where BENZENE failed were primarily due to bugs involving complex inter-procedural data flows that spanned multiple non-adjacent functions, making it difficult for the target function extraction phase to identify the correct mutation site. This points to a natural limitation of the function-granularity mutation strategy.

The under-constrained state mutation strategy is particularly effective for heap and integer overflow bugs, where the crashing condition is determined by a simple scalar relationship (e.g., a buffer size versus an index value). For more complex bug classes such as use-after-free, the approach still works but requires that the UAF-triggering sequence be reproducible within a single function’s execution context.

Overall, the results validate the core insight of the paper: by moving from input-space exploration to state-space ex-

ploration, BENZENE sidesteps the hardest part of behavior generation and achieves both higher accuracy and greater efficiency than prior work.

6 Conclusion

BENZENE presents a compelling and practical approach to automated root cause analysis of software crashes. Its core innovation—under-constrained state mutation at function entry points—elegantly avoids the pitfalls of input-space fuzzing for RCA by directly manipulating program state in memory. The predicate-based crashing condition extraction, combined with a similarity-based ranking algorithm, produces human-interpretable root cause reports that can guide developers directly to the buggy code.

The evaluation results, both in the paper and from exploring the open-source implementation, demonstrate that BENZENE meaningfully advances the state of the art. The tool’s design is also modular enough to serve as a foundation for future extensions, such as automated patch suggestion or integration with static analysis tools. For practitioners working in vulnerability research or software reliability, BENZENE represents a significant step toward making root cause analysis a routine, automated part of the security workflow.

References

- [1] Park, Y., Lee, H., Jung, J., Koo, H., & Kim, H. K. (2024). BENZENE: A Practical Root Cause Analysis System with an Under-Constrained State Mutation. *IEEE Symposium on Security and Privacy (S&P)*. <https://github.com/zer0fall/BENZENE>
- [2] Luo, X., et al. (2021). AURORA: Statistical Crash Analysis for Automated Root Cause Explanation. *USENIX Security Symposium*.
- [3] Blazytko, T., et al. (2022). ARCUS: Symbolic Root Cause Analysis of Exploits in Production Systems. *USENIX Security Symposium*.
- [4] Narayanasamy, S., et al. (2005). Cooperative Bug Isolation. *Proceedings of the 2005 ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*.
- [5] Serebryany, K., et al. (2012). AddressSanitizer: A Fast Address Sanity Checker. *USENIX Annual Technical Conference (ATC)*.